

Yuqiang Xia

+86 17820356950
Guangdong, China
17820356950@139.com
xiayuqiang.xya@alibaba-inc.com
littlexia4-creator.github.io

OBJECTIVE

C/C++ engineer with about 4 years of experience in real-time audio-visual communication and large-scale build infrastructure. Architected a Reclient + NativeLink distributed build that cut Chromium build times by 75% (6h → 1.5h), and hardened RTC media servers against weak networks. Seeking full-time C/C++ / build-engineering roles in China.

Education

Bachelor of Electronic Information Engineering, South China Agricultural University 2019 - 2023
Relevant Coursework: Operating Systems, Data Structures and Algorithms, and Object-Oriented Programming.

SKILLS

Languages (spoken)	English, Mandarin, Cantonese
Programming	C/C++ (C++17/20), C#, Python, Shell
C++ Libraries	Boost, Poco, Google Test, Chromium
Build / Infra	Reclient, NativeLink, ccache, GN/Ninja, CMake, Conan2, msbuild, VS2026, Git/Gerrit, D
Audio & Video	WebRTC, NetEq, video FEC, RTSP, H.264/265, PCM mixing, ffmpeg
Network	VOIP/SIP, RTP, RTCP, TCP, UDP, HTTP, WebSocket, gRPC
Debug / Perf	GDB, Valgrind, ASan, CPU flame graphs, tcpdump, glibc/ptmalloc2 internals, memory-le
Other	multi-threading
AI Dev Tools	Claude Code CLI, Codex CLI, Cursor, OpenClaw

EXPERIENCE

C++ Browser Engine & Build Systems Engineer Jan 2026 - Present
Quark PC / Qwen PC, Alibaba Token Hub *Guangdong, China*

- Architected a **Reclient + NativeLink distributed build** for the Chromium-based Quark/Qwen browser across macOS and Windows, cutting full-build time from **6+ hrs** → **~1.5 hrs (75%)**.
- Established AI-assisted engineering practices: authored agent skill docs and operational runbooks (OpenClaw / Claude Code) standardizing the team's build + caching workflow.

C++ Engineer Jun 2023 - Jan 2026
CVTE *Guangdong, China*

- Enhanced media-server audio anti-jitter: tolerance **10ms** → **50ms**, packet-loss resistance **15%** → **40%** via NetEq.
- Led automated video-conferencing test system — saved **¥2M** in labor cost.
- Recognized with the **Annual Patent Tech-Innovation Award**, **Quality Improvement Award**, and **2nd prize** in the CVTE Micro-Innovation Competition.

C++ Engineer Intern Jun 2022 - Jun 2023
CVTE *Guangdong, China*

- Ramped up on **WebRTC** real-time media; began building the **Virtual Video Conferencing Terminal** automated test system (continued into the full-time role — see Projects).
- Awarded **Excellent Intern A+** performance.

PROJECTS

Quark PC / Qwen PC Distributed Build System

2026.01 - Present

Build-and-cache infrastructure for a Chromium-based desktop browser (Quark PC / Qwen PC) on macOS and Windows. Owned distributed-build architecture, remote caching, and build stability.

Key Responsibilities & Achievements:

1. **Distributed Build Architecture:** Designed and deployed a Reclient + NativeLink system; brought up macOS (arm64) and Windows (x86_64) toolchains and enabled distributed builds by default. Reduced full build from **6+ hrs to ~1.5 hrs (75%)**.
2. **Platform Compatibility:** Worked around the Windows **8192-char command-line limit** by rewriting toolchain invocations to relative paths; engineered the **ccache + Reclient chained integration**; fixed Reclient scan-server failures on V8 (s390 linux-only headers) and the Dawn graphics module.
3. **Remote Caching:** Integrated a **Redis-backed shared remote cache** for ccache across both local builds and the trybot CI cluster. Lifted trybot cluster cache-hit rate from **50–60% to 80%+**; on a warm cache hit, full builds dropped from **3.5 h → 20 min (Windows)** and **<2 h → <10 min (macOS)**, and trybot full-verification fell from a **4 h peak → 1.3 h**.
4. **Local Caching:** Fixed -fmodules cacheability on the macOS Qwen-144 branch so ccache could cache modular compiles, cutting **uncacheable compiler calls from 89% to <1%** and sharply speeding incremental local builds.
5. **Build Stability:** Root-caused intermittent missing-header failures (worsening with higher Ninja -j) to incomplete/circular **GN dependencies**; added correct deps and pruned redundant headers across multiple hardening rounds.
6. **Build Telemetry:** Instrumented the pipeline to report build outcomes, CPU/branch, network state (Wi-Fi vs. wired, incl. USB-LAN detection via carrier status), and ccache stats — driving adaptive strategy (racing exec + resource-fraction tuning off-wire).

Video Conference MCU/SFU Streaming Media Service

2024.01 - 2025.02

Core Real-Time Communication (RTC) platform consisting of MCU servers for mixed-stream forwarding and SFU servers for selective forwarding. Owned server architecture optimization, performance improvement, cross-platform adaptation, and feature development.

Key Responsibilities & Achievements:

1. **Audio Weak-Network Resistance:** Replaced the MCU's traditional Jitter Buffer with WebRTC **NetEq**, resolving compilation and integration challenges. Enhanced anti-jitter tolerance from **10ms → 50ms**, improved packet-loss resistance from **15% → 40%**, and raised the subjective MOS score of MCU audio output.
2. **High-Availability Design:** Split signaling, media, and TURN into independent microservices (Docker + K8s autoscaling); computed weighted node load from CPU usage and system load-average to route new sessions to the least-loaded media node; leveraged K8s health probes for automatic failover and restart.
3. **Transmission-Link Optimization:** Reworked MCU↔SFU cascade audio transcoding from “unconditional full transcode to Opus” to **on-demand transcoding**, eliminating redundant encoding overhead for already-supported audio formats.
4. **Cross-Platform Optimization:** Led the whole C++ service port from **x86 → ARM**, addressing key compilation issues. Designed and built a dual-architecture (ARM/x86) CI/CD pipeline; optimized Jenkins workflows and Conan package management, reducing full compile time from **40min → 15min**.
5. **Real-Time Speaker Detection:** Implemented the real-time speaker-detection function in SFU servers by parsing extension fields in RTP headers, calculating PCM energy values, and applying intelligent anti-shake callbacks.

Virtual Video Conferencing Terminal (Automated Testing System)

2023.01 - 2023.12

This system simulates large-scale video-conferencing terminals (1000+) joining in batch to test the WebRTC media server. It liberates testers, expands test coverage, and accelerates the iteration cycle — saving the team approximately **~¥2M in labor cost and ¥5M in equipment cost** annually, and reducing the rate of P0-level (catastrophic) issues at customer sites from **5% → 0%**.

Key Responsibilities & Achievements:

1. **Core Architecture:** Independently designed a four-layer architecture for SIP-based virtual terminals — a business layer (control commands over pipes/IPC), an instance layer (binds signaling and media), a signaling layer

(SIP signaling and SDP negotiation based on the reSIProcate library), and a media layer (RTP stream transmission and control); used mmap-backed shared memory for inter-process data.

2. **Cross-Platform Design:** Adopted Poco cross-platform technology to ensure terminal compatibility with Windows, Linux, and macOS; implemented automated builds on Jenkins using CMake/Conan.

3. **Simulation Replay Solution:** Adopted a file-playback mode that directly reads pre-recorded .rtp/.pcap files and sends RTP streams, achieving lightweight operation and high performance (supporting hundreds of simulated terminals per server). The media layer supports control commands such as mute and I-frame retransmission, and is compatible with multiple video formats (H.263/H.264/H.265) and audio formats (AAC/Opus/G.719).

4. **Engineering Deployment:** Packaged virtual terminals and the scheduling center into Docker images, deployed on a Kubernetes-based DevOps platform. Enabled testers to operate tests via a web interface and R&D personnel to schedule the test system through Python scripts, realizing elastic scheduling and automated execution of test resources.